第 4 章 文字如何变成数字：Token、注意力与语言模型

> 本章目标：不推导公式，也能说清一句话从你按下回车、到模型吐出一个字，中间经过了什么。学完你应该能解释：token 是什么、注意力在做什么、上下文窗口为什么有限、模型为什么会一本正经地胡说。

4.1 Token：模型眼里的"字"

模型不认字，只认整数。第一步是把文本切成小块，每块叫一个 **token**，再查表换成编号，这个过程叫分词（tokenization）。

切法和直觉不一样：

- 英文按**子词**切。`playing` 常被切成 `play` + `ing`，`unbelievable` 常被切成 `un` + `believ` + `able`。这样词表不必收录每个单词的所有变形。
- 中文常见词可能整词算一个 token，生僻字可能被拆成两个（因为它由多个字节组成）。做余量估算时按"一个汉字约 1~2 个 token"粗略估计即可——但这个比例随分词器变化很大，不是定律。
- 空格、换行、标点、JSON 的大括号和引号**全都占 token**。写代码和传结构化数据，token 消耗往往比同长度的中文高不少。

Token 是三个东西的计量单位：**计费**按 token 算，**上下文窗口**按 token 算，**生成速度**也按 token 算。

> 要点：界面上看到的字数和模型实际处理的 token 数不是一回事。做成本估算看 token，不看字数。

4.2 Embedding：把 token 变成向量

编号本身没有意义，`1043` 和 `1044` 之间不存在"相邻"。所以模型给每个 token 配一个**向量**（一长串小数，几百到几千维），这叫 **embedding**。

训练之后，这个空间会呈现语义结构：意思和用法相近的词，向量也接近。"猫"靠近"狗"，远离"水泥"。这些坐标不是人填的，是从海量文本里统计出来的。

一个容易被忽略的细节：同一个词在不同句子里，经过若干层之后，向量会被改写成**依赖上下文**的版本。所以"苹果手机"里的"苹果"和"吃苹果"里的"苹果"，内部表示并不相同。

> 类比：给每个词在一张大地图上定坐标，意思相近的靠在一起。区别是这张地图有几百上千个维度，人画不出来。

4.3 注意力：每个词按相关度向其他词取信息

先看一句话：

> 小明的猫把杯子碰倒了，因为它很调皮。

人读到"它"，知道指的是猫。模型是怎么处理的？在为"它"这个位置计算表示时，它会为句中每个词算一个**相关度分数**，再按分数把其他词的信息**加权混合**进来：



```
| ② 分数归一化成权重，越相关权重越大
▼
猫 0.50    调皮 0.20    杯子 0.15    碰倒 0.08    小明 0.05    ……（示意，非真实数值）
|
| ③ 按权重把每个词的内容（"值"）加权求和
▼
"它"的新表示 ≈ 主要带着"猫"的信息，混入少量"调皮""杯子"
|
▼
进入下一层，继续被改写
```

一句话概括：**注意力 = 每个词按相关度，从其他词那里取信息。**"查询 / 键 / 值"只是这个动作的三个角色，不是三个新概念。

> 类比：去图书馆找资料。"猫"这个主题是你心里的**查询**；每本书脊上的分类标签是**键**，用来和你比对；书里的正文是**值**，真正被取走的内容。你按标签匹配度决定每本书借多少内容回来。

这里要说一句实话：注意力权重反映的是**训练数据里的统计倾向**，不是逻辑推理。上面那句里，"小明"其实也可能是调皮的，语料里确实有这种用法。模型的"理解"是概率意义上的，不是符号推理意义上的。

4.4 多头注意力：多组视角并行

一次注意力只能看一种"相关"。Transformer 同时跑很多组（**多头**），每组有自己的查询/键/值变换，相当于从不同角度扫同一句话：一组可能盯住"它"最近的名词，一组可能盯住"碰倒—杯子"的动宾关系，一组可能盯住整体语气。最后把各组输出拼起来。

多头 = 多组不同视角并行，再把结果合并。 具体哪一组负责什么，是训练自己长出来的，不同模型差别很大。研究者能观察到一些规律，但别把它当成可以手动拨的开关。

4.5 Transformer 块：同一种结构重复堆叠

一层由四件事组成：



- **注意力**负责词与词之间交换信息。
- **前馈网络**在单个位置上做加工，不跨位置。
- **残差**（那条 `(+)` 旁路）让每层只学"增量"，信号能一路传下去，深层网络才训得动。
- **层归一化**稳住数值尺度，防止训练发散。

大模型就是把这一块**堆很多层**，参数主要花在注意力和前馈网络的矩阵上。具体排列（先归一化还是后归一化、用哪种归一化）各家有差异，但"注意力 + 前馈 + 残差 + 归一化"这个骨架是共同的。

4.6 位置编码：顺序必须额外告诉模型

注意力有个副作用：它对**顺序不敏感**。把"猫追狗"和"狗追猫"的词打乱喂进去，加权求和的结果一样。这显然不行。

所以位置信息要**额外注入**：给每个位置一个可区分的位置向量，加到词向量上，或在注意力里加入相对位置信息。正弦编码、可学习位置向量、相对位置、旋转位置编码都属于这类做法，原理都是"让模型知道谁在前谁在后、隔多远"。

4.7 自回归生成：一次只猜下一个 token

语言模型做的事只有一件：**给定前面所有 token，输出下一个 token 的概率分布，采样一个，接回末尾，再来一遍。**



一个示意性的概率分布（**数字为讲解编造，不是真实模型输出**）：

候选下一个 token	概率
好	0.42

候选下一个 token	概率
热	0.18
冷	0.09
晴	0.07
不错	0.06
其他	0.18

温度（temperature） 调节这个分布的尖锐程度。调低，概率更集中，几乎总选"好"，稳定但保守；调高，分布被拉平，"冷""晴"也有机会，更多样，也更容易跑偏。

top-p（核采样） 是另一道闸：只保留累计概率达到 p 的那一小撮候选，再在里面采样。p 小则候选少、更稳；p 大则放得更开。通常留一个用默认值即可，不必同时细调两个。

4.8 同一个问题，两次答案为什么不一样

三个原因，按重要性排：

1. **采样有随机性**。只要不是每步都取概率最高的那个，路径就会分叉。分叉一次，后面基于它的所有 token 都跟着变。
2. **服务端不是严格确定性的**。浮点求和顺序、批处理拼包、不同硬件，都可能让概率出现微小差异；模型版本更新也会改变行为。
3. **上下文可能变了**。当前时间、检索到的网页、工具返回结果、对话历史，每次都可能不同。

所以即便把温度调到 0，也不能保证两次输出逐字相同。**"可复现"在 LLM 上是需要工程手段去逼近的目标，不是默认属性。** 这也是第五章强调"把状态写进文件和 git"、而不是"靠模型记住"的原因。

4.9 上下文窗口：有限且会衰减的工作记忆

上下文窗口 = 一次请求里模型能同时看到的 token 总量上限，包括系统提示、历史对话、你贴的文件、工具返回结果，以及要生成的回复。

三个现实后果：

- **硬上限**。超了就装不下，产品要么报错，要么自动截断/压缩——被截掉的部分模型真的看不见。
- **要花钱**。输入通常按量计费，而且每次对话都要把整个上下文重新送一遍，所以长对话越聊越贵。
- **影响质量**。上下文极长时，中间部分容易被稀释：开头和结尾被注意得更多，中段容易被略过。不同模型差异很大，但"塞得越多不等于答得越好"是普遍现象。

> 要点：上下文不是硬盘，是内存——而且很贵、会衰减。重要的东西要落到文件里，不要指望它一直"记得"。

4.10 训练的三个阶段

预训练（pre-training）：拿海量文本反复做"遮住下一个词让它猜"，猜错就调参数。做够久之后，语法、常识、代码模式、世界知识的统计结构都压进了权重。这一步最贵，产出的是一个"续写能力极强但不会好好对话"的基座模型——你给它一句话，它可能接着往下写，而不是回答你。

监督微调（SFT）：用大量"问题—理想回答"的示范对，教它对话的格式和基本行为：该答就答、该拒绝就拒绝、该调工具就按格式输出。

偏好对齐（RLHF 等）：让人对多个回答排序，训练一个"打分器"，再用它优化模型，让它更倾向人类喜欢的风格——有用、清楚、不啰嗦、不乱来。这一阶段塑造的是**风格和倾向**，不是知识。

三者合起来的效果：模型既会"接着往下写"，又被调教成"照着指令作答"。

4.11 训练与推理，别搞混

	训练	推理
做什么	按误差调整权重	权重固定，做前向计算
频率	一次，成本极高	每次请求都做
和你的关系	与你无关	你的每次对话就是它
谁在做	模型厂商	你用的产品 / API

最重要的一条：**你聊天时"教会"它的东西，不会写回权重。** 它只是在当前上下文里顺着演。关掉会话，什么都不会留下。想让行为长期改变，只能靠提示词、项目指令文件、外部记忆——这些都属于harness。

4.12 为什么模型会"胡说"

因为它的训练目标是**生成看起来合理的下一个 token**，不是**说出真话**。这两件事高度相关，但不是同一件事。

- 遇到不熟的问题，它仍然要给出一个"最可能的续写"。编一个格式正确、语气自信、细节自洽的答案，在统计上完全说得通。
- 训练语料里的错误、过时信息、虚构内容，同样会被学进去。
- 它没有内置的"这条我不知道"检测器。不确定性不会自动变成一句"我不确定"。
- 编造往往比坦承无知更流畅，而流畅正是它被优化的方向。

所以**幻觉不是偶发故障，是这套机制的副产品**。它不会因为"你再仔细想想"就消失。真正管用的是外部校验：跑一遍代码、打开文件看一眼、查资料核对、要求给出可验证的依据。

4.13 "世界知识"的边界在哪

模型的知识不是数据库，而是**训练文本里统计规律的压缩**。由此有确定的边界：

- **有截止时间。** 训练完成之后的事它不知道。问最新软件版本、今天的新闻、刚出的法规，它可能给出过时答案，语气一样自信。
- **没有实时信息。** 除非产品给它接了搜索或工具，否则它没法"上网看看"。
- **不知道你的私有内容。** 没读过的文件、你的项目代码、内部文档，它一概不知——除非你把内容放进上下文。
- **精确事实最不可靠。** 数字、日期、人名、API 签名、文件路径、引用出处，是幻觉高发区。越是要求精确，越要核对。
- **冷门领域更差。** 语料里出现得少，统计规律就弱。

> 要点：把模型当成"读过很多书、记性一般、特别会说话、从不承认自己没读过"的助手。它的自信程度和正确率之间，没有稳定关系。

本章小结

一句话串起来：文本 → 切成 token → 查成向量 → 加上位置 → 多层"注意力 + 前馈"反复改写 → 输出下一个 token 的概率 → 采样 → 接回末尾再来一遍。训练让它从"会续写"变成"会对话"；推理只做前向计算，不改权重；上下文窗口是有限且会衰减的工作记忆；幻觉是机制的副产品，不是偶发 bug。理解了这些，第五章的工程规矩就不再是玄学，而是有原因的。

课后题

1. 找一个英文单词和一个中文词，用任意产品的 token 计数功能切一下，记录各被切成几个 token，和你的直觉对比。

2. 用"小明的猫把杯子碰倒了，因为它很调皮"自己画出"它"的注意力权重示意。若改成"小明把杯子碰倒了，因为他很调皮"，权重会怎么变？为什么？
3. 同一个问题连续问三次，记下差异，用"采样随机性"解释它们为什么不同。
4. 找一个你熟悉、但模型很可能不知道的冷门事实去问它，观察它有没有编造，以及有没有表现出"不确定"。
5. 解释为什么"我在对话里教了它我的名字，它还是会在新会话里忘掉"。
6. （开放）有种观点认为"模型只是概率续写，所以不可能真正理解"。结合本章内容说说你同意到什么程度、理由是什么；再说说这个判断对"该不该信它的输出"有什么实际影响。